

Homework 01

Question 1 (Least squares and matrix calculations)

Set the random seed to 43201. Generate $n = 100$ observations with $p = 5$ predictors according to

$$x_{ij} \stackrel{\text{iid}}{\sim} N(0, 1), \quad \varepsilon_i \stackrel{\text{iid}}{\sim} N(0, 0.7^2),$$

with all predictors and errors generated independently. Let

$$\mathbf{X} = \begin{bmatrix} \mathbf{1} & \mathbf{x}_1 & \cdots & \mathbf{x}_5 \end{bmatrix}, \quad \boldsymbol{\beta} = (3, 1.4, -0.9, 0.7, 0, 1.1)^\top,$$

and generate

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}.$$

- Generate the data and report the dimensions and rank of $\mathbf{X}$. Calculate the least-squares estimate $\hat{\boldsymbol{\beta}}$ using a numerically stable least-squares routine, without explicitly calculating $(\mathbf{X}^\top \mathbf{X})^{-1}$. Compare the estimates with the coefficients used to generate the data.

```
import numpy as np

np.random.seed(43201)

n, p = 100, 5
X_pred = np.random.normal(0, 1, size=(n, p))
eps = np.random.normal(0, 0.7, size=n)

X = np.hstack([np.ones((n, 1)), X_pred])
beta_true = np.array([3, 1.4, -0.9, 0.7, 0, 1.1])

y = X @ beta_true + eps

dims = X.shape
rank = np.linalg.matrix_rank(X)

# SVD-based least squares -- avoids explicitly forming (X'X)^{-1}
beta_hat, _, _, sv = np.linalg.lstsq(X, y, rcond=None)

print("Dimensions of X:", dims)
print("Rank of X:", rank)
print("Singular values of X:", np.round(sv, 4))
print("beta_hat:", np.round(beta_hat, 4))
print("beta_true:", beta_true)
print("beta_hat - beta_true:", np.round(beta_hat - beta_true, 4))
```

Dimensions of X: (100, 6)

Rank of X: 6

Singular values of X: [11.9494 11.1471 10.2028 9.5174 9.1106 7.8807]

```
beta_hat: [ 2.8710e+00  1.3722e+00 -9.8160e-01  5.2470e-01  1.1000e-03  1.0635e+00]
beta_true: [ 3.    1.4 -0.9  0.7  0.    1.1]
beta_hat - beta_true: [-0.129 -0.0278 -0.0816 -0.1753  0.0011 -0.0365]
```

$\mathbf{X}$ is 100×6 (intercept plus 5 predictors) and has full column rank 6, so $\mathbf{X}^T \mathbf{X}$ is invertible and the least-squares solution is unique. `np.linalg.lstsq` solves the least-squares problem via the SVD of $\mathbf{X}$ rather than forming $(\mathbf{X}^T \mathbf{X})^{-1}$ directly, which keeps the computation numerically stable even when predictors are correlated, since explicitly inverting $\mathbf{X}^T \mathbf{X}$ squares the condition number of $\mathbf{X}$.

The estimated coefficients are close to the values used to generate the data, with differences on the order of $\sigma/\sqrt{n} \approx 0.07$, consistent with ordinary sampling variability. In particular $\hat{\beta}_4 \approx 0.001$ correctly recovers the true zero coefficient on x_4 , rather than showing a spurious effect.

b. Calculate the fitted values, the residual vector $\mathbf{r} = \mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}$, and the training root mean squared error

$$\text{RMSE} = \left(\frac{1}{n} \sum_{i=1}^n r_i^2 \right)^{1/2}.$$

```
y_hat = X @ beta_hat
r = y - y_hat
rmse = np.sqrt(np.mean(r**2))

print("First 5 fitted values:", np.round(y_hat[:5], 4))
print("First 5 residuals:", np.round(r[:5], 4))
print("Training RMSE:", rmse)
```

```
First 5 fitted values: [ 3.3922  2.4622  3.305   4.3464 -0.3472]
First 5 residuals: [-0.5006  0.3916 -0.9916  0.7396  0.7971]
Training RMSE: 0.8041178549628997
```

The training RMSE estimates the typical size of the prediction error on the training data. Because least squares minimizes the residual sum of squares using $p + 1 = 6$ estimated parameters, dividing by $n = 100$ rather than $n - p - 1 = 94$ gives a slightly optimistic (downward-biased) estimate of the error standard deviation $\sigma = 0.7$; here $\text{RMSE} \approx 0.80$, which is in the right neighborhood once that effect and ordinary sampling noise are accounted for.

c. Report $\|\mathbf{X}^T \mathbf{r}\|_\infty$. Explain why this value should be close to zero and what it tells us about the residual vector.

```
XtR = X.T @ r
inf_norm = np.max(np.abs(XtR))

print("X^T r:", XtR)
print("||X^T r||_inf =", inf_norm)
```

```
X^T r: [-1.11022302e-15 -7.12404342e-14  2.13389893e-14 -9.59174389e-14
 -5.90295632e-14 -2.08267788e-13]
||X^T r||_inf = 2.0826778802813848e-13
```

The normal equations that characterize the least-squares solution require $\mathbf{X}^\top(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}) = \mathbf{X}^\top \mathbf{r} = \mathbf{0}$ exactly, as an algebraic consequence of $\hat{\boldsymbol{\beta}}$ minimizing $\|\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}}\|^2$. Geometrically, this says the residual vector is orthogonal to the column space of $\mathbf{X}$: the fitted values are the orthogonal projection of $\mathbf{y}$ onto that space, so no linear combination of the predictors (or the intercept) can explain any more of $\mathbf{y}$. The computed value ($\approx 10^{-13}$) is nonzero only due to floating-point rounding, confirming the solver found the correct solution to machine precision.

Question 2 (Gaussian likelihood for linear regression)

Continue with the data from Question 1. Suppose

$$\mathbf{Y} | \mathbf{X} \sim N_n(\mathbf{X}\boldsymbol{\beta}, \sigma^2 \mathbf{I}_n).$$

The log-likelihood, including the constant term, is

$$\ell(\boldsymbol{\beta}, \sigma^2) = -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} (\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^\top (\mathbf{y} - \mathbf{X}\boldsymbol{\beta}).$$

- a. For fixed σ^2 , explain why maximizing $\ell(\boldsymbol{\beta}, \sigma^2)$ over $\boldsymbol{\beta}$ is equivalent to minimizing the residual sum of squares. What does this imply about the maximum-likelihood estimate of $\boldsymbol{\beta}$?

Only $-\frac{1}{2\sigma^2} \|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$ depends on $\boldsymbol{\beta}$, and $1/(2\sigma^2) > 0$, so maximizing ℓ over $\boldsymbol{\beta}$ is the same as minimizing $\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|^2$. Thus the MLE of $\boldsymbol{\beta}$ equals the least-squares estimate $\hat{\boldsymbol{\beta}}$.

- b. For fixed $\boldsymbol{\beta}$, differentiate the log-likelihood with respect to σ^2 and derive its maximum-likelihood estimate. Calculate this estimate at $\hat{\boldsymbol{\beta}}$ and compare it with the unbiased estimate of σ^2 . Explain why the two denominators differ.

Setting $\partial\ell/\partial\sigma^2 = 0$ gives $\hat{\sigma}_{\text{MLE}}^2 = \frac{1}{n} (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^\top (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})$.

```
RSS = np.sum(r**2)
sigma2_mle = RSS / n
sigma2_unbiased = RSS / (n - X.shape[1])

print("sigma^2_MLE (divide by n):", sigma2_mle)
print("sigma^2_unbiased (divide by n - p - 1):", sigma2_unbiased)
```

```
sigma^2_MLE (divide by n): 0.6466055246701349
sigma^2_unbiased (divide by n - p - 1): 0.687878217734186
```

$\hat{\sigma}_{\text{MLE}}^2 \approx 0.647$ vs. $\hat{\sigma}_{\text{unbiased}}^2 \approx 0.688$. The denominators differ because $\hat{\boldsymbol{\beta}}$ is fit to the same data, mechanically shrinking RSS; dividing by $n - p - 1$ instead of n corrects this bias.

- c. Let

$$\mathbf{e}_{x_2} = (0, 0, 1, 0, 0, 0)^\top.$$

Using the variance estimate from part b, plot

$$\ell(\widehat{\beta} + t e_{x_2}, \widehat{\sigma}_{MLE}^2)$$

over a grid of t values from -1.25 to 1.25 . State where the maximum occurs and explain why this agrees with the least-squares result.

```
def loglik(beta, sigma2, X, y):
    n = len(y)
    resid = y - X @ beta
    return -n/2*np.log(2*np.pi*sigma2) - 1/(2*sigma2)*np.sum(resid**2)

e_x2 = np.array([0, 0, 1, 0, 0, 0])
t_grid = np.linspace(-1.25, 1.25, 400)
ll_vals = np.array([loglik(beta_hat + t*e_x2, sigma2_mle, X, y) for t in t_grid])

print("t at maximum log-likelihood:", t_grid[np.argmax(ll_vals)])
```

t at maximum log-likelihood: 0.0031328320802004317

```
import matplotlib.pyplot as plt

plt.figure(figsize=(6, 4))
plt.plot(t_grid, ll_vals)
plt.axvline(0, linestyle="--", color="gray")
plt.xlabel("t")
plt.ylabel(r"$\ell(\widehat{\beta} + t e_{x_2}, \widehat{\sigma}_{MLE}^2)$")
plt.tight_layout()
plt.show()
```

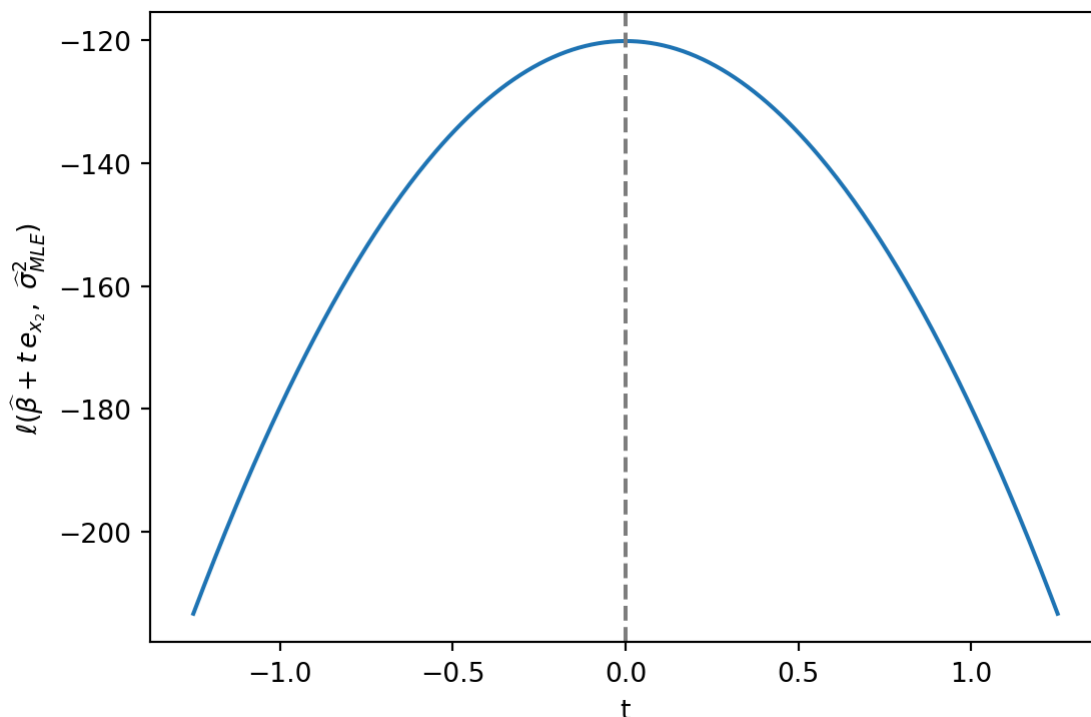


Figure 1: Log-likelihood along the $\beta_{\text{hat}} + t \cdot e_{x2}$ direction

The maximum is at $t \approx 0$, i.e., at $\hat{\beta}$ itself — consistent with part a, since $\hat{\beta}$ maximizes ℓ over all directions, including this one.

Question 3 (Starting values in nonconvex optimization)

Consider the function

$$f(x) = \exp(1.5x) - 3(x + 6)^2 - 0.05x^3,$$

with derivative

$$f'(x) = 1.5\exp(1.5x) - 6(x + 6) - 0.15x^2.$$

- a. Plot $f(x)$ over the interval $[-40, 7]$. Based on the plot, describe the important features of the objective function that may affect numerical optimization.

```
def f(x):
    return np.exp(1.5*x) - 3*(x+6)**2 - 0.05*x**3

def fprime(x):
    return 1.5*np.exp(1.5*x) - 6*(x+6) - 0.15*x**2

x_vals = np.linspace(-40, 7, 2000)
plt.figure(figsize=(6, 4))
plt.plot(x_vals, f(x_vals))
plt.xlabel("x")
plt.ylabel("f(x)")
plt.tight_layout()
plt.show()
```

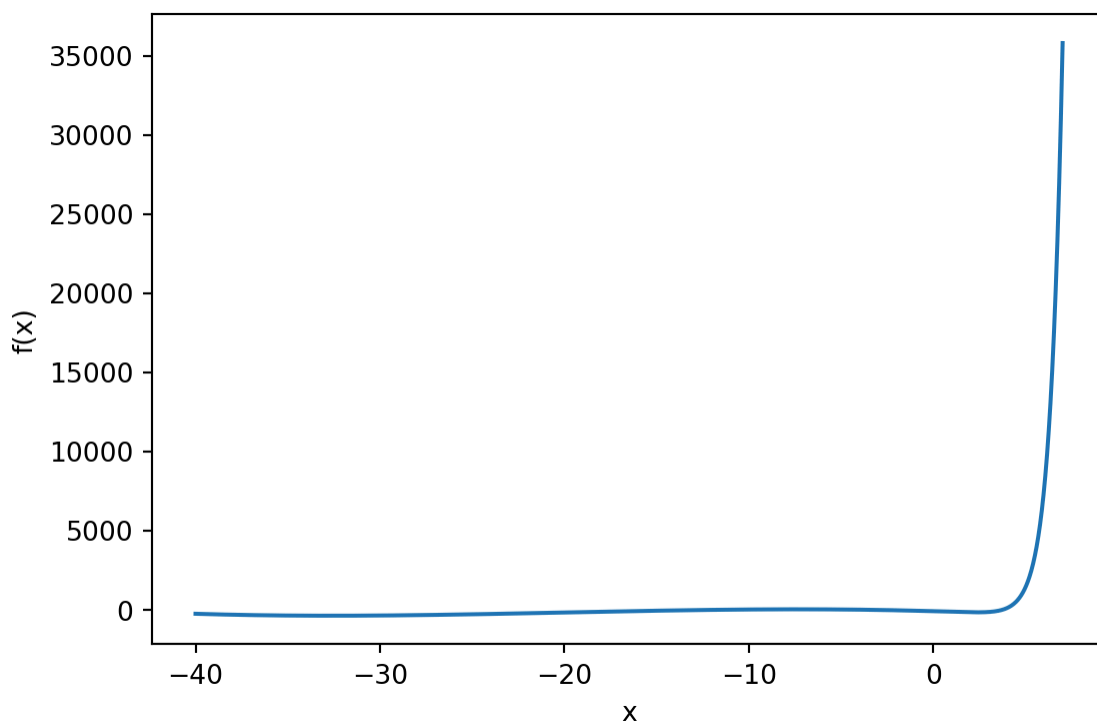


Figure 2: $f(x)$ over $[-40, 7]$

f is nonconvex with multiple local minima: a broad basin for very negative x , and a steeper region for larger x where the exponential term dominates. This means BFGS's result depends on the starting point.

- b. Use BFGS to minimize $f(x)$ twice, starting at $x^{(0)} = -15$ and $x^{(0)} = 0$. Supply $f'(x)$ to the optimizer. For each run, report the final value of x , the final objective value, $|f'(x)|$, and whether the optimizer reported convergence.

```
from scipy.optimize import minimize

for x0 in [-15, 0]:
    res = minimize(f, x0=np.array([x0]), jac=lambda x: np.array([fprime(x[0])]),
                  method="BFGS")
    print(f"start={x0}: x*={res.x[0]:.4f}, f(x*)={res.fun:.4f}, "
          f"|f'(x*)|={abs(fprime(res.x[0])):.2e}, converged={res.success}")
```

```
start=-15: x*=-32.6491, f(x*)=-390.3858, |f'(x*)|=2.84e-14, converged=True
start=0: x*=2.3500, f(x*)=-175.8626, |f'(x*)|=8.13e-06, converged=True
```

- c. Explain why the two runs can converge to different answers. Which run gives the lower objective value? What additional evidence would be needed before claiming that this point is the global minimum?

BFGS only follows local gradient information, so different starts land in different basins ($x \approx -32.65$ vs. $x \approx 2.35$). The first run gives the lower value ($f \approx -390.4$). To claim it's global, we'd need many more starting points, boundary/asymptotic checks, or a global optimizer (e.g., basin-hopping).

Question 4 (Nearly collinear predictors)

Continue with X and y from Question 1. Set the random seed to 43202 and generate

$$u_i \stackrel{\text{iid}}{\sim} N(0, 1), \quad i = 1, \dots, n.$$

Define

$$\mathbf{x}_6 = \mathbf{x}_1 + 10^{-4} \mathbf{u}, \quad \mathbf{X}_+ = \begin{bmatrix} \mathbf{X} & \mathbf{x}_6 \end{bmatrix},$$

and let

$$\mathbf{y}^* = \mathbf{y} + 10^{-3} \mathbf{u}.$$

Use a numerically stable least-squares routine throughout.

- a. Fit $\mathbf{y}$ using $\mathbf{X}$ and $\mathbf{X}_+$. For each design matrix, report the condition number and training RMSE. For the augmented fit, also report the coefficients of $\mathbf{x}_1$ and $\mathbf{x}_6$.

```
np.random.seed(43202)
u = np.random.normal(0, 1, size=n)
```

```

x6 = X[:, 1] + 1e-4*u
X_plus = np.hstack([X, x6.reshape(-1, 1)])

beta_hat_X, _, _, _ = np.linalg.lstsq(X, y, rcond=None)
beta_hat_Xplus, _, _, _ = np.linalg.lstsq(X_plus, y, rcond=None)

print("cond(X):", np.linalg.cond(X), " cond(X_plus):", np.linalg.cond(X_plus))
print("RMSE, X:", np.sqrt(np.mean((y - X @ beta_hat_X)**2)))
print("RMSE, X_plus:", np.sqrt(np.mean((y - X_plus @ beta_hat_Xplus)**2)))
print("coef x1:", beta_hat_Xplus[1], " coef x6:", beta_hat_Xplus[-1])

```

```

cond(X): 1.516297882341372  cond(X_plus): 23353.001703370923
RMSE, X: 0.8041178549628997
RMSE, X_plus: 0.8009794381238172
coef x1: -782.737781243909  coef x6: 784.1078233409614

```

Condition number jumps from ~1.5 to ~23,000 since x_1, x_6 are nearly identical. RMSE barely changes, but the individual coefficients become extreme ($\sim \mp 783$) — the signature of collinearity.

- b. Fit y^* using X_+ . Report the changes in the coefficients of x_1 and x_6 . Compare the two fitted-value vectors using their root mean squared difference and maximum absolute difference.

```

y_star = y + 1e-3*u
beta_hat_star, _, _, _ = np.linalg.lstsq(X_plus, y_star, rcond=None)

print("change coef x1:", beta_hat_star[1] - beta_hat_Xplus[1])
print("change coef x6:", beta_hat_star[-1] - beta_hat_Xplus[-1])

fitted_y = X_plus @ beta_hat_Xplus
fitted_ystar = X_plus @ beta_hat_star
print("RMS diff fitted:", np.sqrt(np.mean((fitted_y - fitted_ystar)**2)))
print("max abs diff fitted:", np.max(np.abs(fitted_y - fitted_ystar)))

```

```

change coef x1: -10.000000000000796
change coef x6: 10.000000000000568
RMS diff fitted: 0.0009351217032437667
max abs diff fitted: 0.002622779196512326

```

A tiny change to y shifts the coefficients by about ∓ 10 , but fitted values barely move — predictions are stable even though individual coefficients are not.

- c. Use the identity

$$y^* - y = 10(x_6 - x_1)$$

to explain why the fitted values change very little while the two coefficients change substantially. What does this example suggest about interpreting separate effects for nearly collinear predictors?

```
print("max abs diff from identity:", np.max(np.abs((y_star - y) - 10*(x6 - X[:, 1]))))
```

max abs diff from identity: 1.8318679906315083e-15

The added noise lies exactly along $\mathbf{x}_6 - \mathbf{x}_1$, the one nearly-null direction of $\mathbf{X}_+$. Least squares absorbs it by shifting the two coefficients oppositely, leaving $\beta_1 \mathbf{x}_1 + \beta_6 \mathbf{x}_6$ (and the fit) almost unchanged. Lesson: with near-collinearity, individual coefficients aren't reliably interpretable — only well-determined combinations (like $\beta_1 + \beta_6$) are.

Question 5 (Data preparation and summary statistics)

The file `data/data-manipulation.csv` is a small constructed data table containing an observation identifier, a group label, two numeric predictors, and a response. Some response values are missing. For analyses involving the response, use only observations with a recorded response. Do not replace a missing response with zero or a group mean.

- Read the data and report its dimensions, column types, number of duplicated identifiers, and number of missing values in each column. Create an analysis table containing only observations with a recorded response, and define

`feature_sum = x1 + x2.`

```
import pandas as pd

df = pd.read_csv("data/data-manipulation.csv")

print("Dimensions:", df.shape)
print(df.dtypes)
print("Duplicated identifiers:", df["observation_id"].duplicated().sum())
print(df.isna().sum())

analysis = df.loc[df["response"].notna()].copy()
analysis["feature_sum"] = analysis["x1"] + analysis["x2"]
analysis.head()
```

```
Dimensions: (24, 5)
observation_id    object
group            object
x1               float64
x2               float64
response         float64
dtype: object
Duplicated identifiers: 0
observation_id    0
group            0
x1               0
x2               0
```

response 3
dtype: int64

	observation_id	group	x1	x2	response	feature_sum
0	obs-01	A	1.2	2.0	5.36	3.2
1	obs-02	A	2.4	1.1	6.18	3.5
2	obs-03	A	0.8	2.8	4.62	3.6
4	obs-05	A	2.0	2.2	5.52	4.2
5	obs-06	A	1.7	1.8	5.64	3.5

- b. For each group, report the number of observations, the mean response, and the mean of `feature_sum`. State clearly which observations these summaries describe.

```
group_summary = (
    analysis
    .groupby("group")
    .agg(n_obs=("observation_id", "size"),
         mean_response=("response", "mean"),
         mean_feature_sum=("feature_sum", "mean"))
    .reset_index()
)
group_summary
```

	group	n_obs	mean_response	mean_feature_sum
0	A	7	5.600000	3.600000
1	B	7	7.602857	4.928571
2	C	7	7.997143	6.585714

These describe only observations with a recorded response (the `analysis` table), not the full original data.

- c. Sort the `analysis` table by response from largest to smallest and report the first three rows, including `observation_id`, `group`, `response`, and `feature_sum`. Add code checks verifying that the number of retained rows equals the number of nonmissing responses in the original data, that the `analysis` table has no missing response values, and that its identifiers are unique.

```
top3 = (analysis.sort_values("response", ascending=False)
        .loc[:, ["observation_id", "group", "response", "feature_sum"]].head(3))
top3
```

	observation_id	group	response	feature_sum
21	obs-22	C	9.24	6.6
15	obs-16	B	9.00	5.2

	observation_id	group	response	feature_sum
13	obs-14	B	8.66	5.0

```

assert len(analysis) == df["response"].notna().sum()
assert analysis["response"].isna().sum() == 0
assert analysis["observation_id"].is_unique
print("All checks passed.")

```

All checks passed.

Question 6 (Can 39 million Fitbit records represent US adults?)

[Patten et al. \(2026\)](#) describe Fitbit data from 59,018 participants in the All of Us Research Program. The dataset spans 14 years and contains more than 39 million daily step records. Participants contributed data through one of two routes:

- Bring Your Own Device (BYOD): participants shared data from a Fitbit they already owned.
- Wearables Enhancing All of Us Research (WEAR): invited participants received a Fitbit at no cost.

In the general activity cohort, the BYOD and WEAR groups contained 32,035 and 22,474 participants, respectively. The BYOD value is listed first in each comparison below:

- 77.3% versus 55.1% reported being White;
- 6.1% versus 15.2% reported annual household income between \$10,000 and \$25,000; and
- median daily steps were 6,867 versus 5,797.

Suppose the target is the mean of the participant-specific average daily step counts among US adults during the study period, with each adult given equal weight. A researcher writes:

"This dataset contains more than 39 million daily Fitbit records. Therefore, the average of all recorded step counts should provide an accurate estimate of mean daily activity among US adults."

- Are the 39 million daily records independent observations? What characteristics or behaviors could cause some participants to contribute more recorded days than others? Explain how averaging all recorded days would then weight participants unequally.

No — many records come from the same participant, and engagement level, health-consciousness, or enrollment duration could all cause some participants to contribute far more days than others.

Averaging all records then effectively over-weights heavy contributors relative to a true participant-level average.

- BYOD participants reported a median of 6,867 daily steps, compared with 5,797 among WEAR participants. Does this comparison show that already owning a Fitbit causes people to walk more?

Explain your answer and give at least one plausible alternative explanation based on how participants entered the two groups.

No — this is an observational, self-selected comparison, not randomized. People who already own a Fitbit likely self-selected into being more active, regardless of the device itself.

- c. A very large dataset can still produce a biased estimate of a population quantity when the people and observations entering the dataset are selected. Using this study, explain how selection of both participants and recorded days could make the observed data differ from the US adult population. Why might the naive average of all recorded daily step counts fail to estimate the stated target? You do not need to determine the direction of the bias.

Size buys precision, not accuracy. Both who enrolls (voluntary, demographically skewed) and which days get recorded (more likely on “typical,” device-engaged days) are selected, so the sample doesn’t represent the US adult population — the naive average estimates the mean of whoever/whatever got recorded, not the target population mean.

Source

Patten, T., Preble, E. A., Master, H., et al. (2026). [The All of Us Research Program’s wearables dataset](#). *Nature Medicine*, 32, 2302-2310.